

Lista de Exercícios — Programação Orientada a Objetos em C++

Conteúdos abordados: classes, construtores, encapsulamento, métodos acessores, herança, vetores de objetos, matrizes de objetos e tratamento de exceção.

Instruções: Para cada exercício, implemente o programa em C++ utilizando corretamente classes, atributos privados, construtores, métodos acessores e tratamento de exceção. Sempre que o enunciado pedir validação de dados, utilize `throw` para lançar exceções e `try/catch` no programa principal para tratar os erros.

Em todos os exercícios, os atributos das classes devem ser declarados como `private`, e o acesso a eles deve ser feito por meio de métodos públicos. Os construtores devem ser utilizados para inicializar os objetos corretamente, e os métodos `set` devem ser responsáveis por validar os dados recebidos. Sempre que um dado inválido for informado, o programa deve lançar uma exceção usando `throw invalid_argument`.

Questão 1 — Sistema de notas com alunos e monitores

Crie um programa em C++ para controlar as notas de uma turma. O sistema deverá possuir uma classe `Prova`, contendo os atributos privados `disciplina` e `nota`, e uma classe `Aluno`, contendo os atributos privados `matricula`, `nome` e um vetor com exatamente **3 objetos da classe `Prova`**. A classe `Prova` deve possuir construtor padrão, construtor com parâmetros, métodos `setDisciplina`, `setNota`, `getDisciplina` e `getNota`, sendo que o método `setNota` deve lançar uma exceção caso a nota seja menor que 0 ou maior que 10. A classe `Aluno` deve possuir construtor padrão, construtor com parâmetros, métodos `setMatricula`, `setNome`, `getMatricula`, `getNome`, `setProva`, `getProva`, `calcularMedia`, `getSituacao` e `imprimir`.

O método `calcularMedia` deve calcular a média das 3 provas, o método `getSituacao` deve retornar `"Aprovado"` caso a média seja maior ou igual a 6 e `"Reprovado"` caso contrário, e o método `imprimir` deve mostrar todos os dados do aluno e suas provas. Além disso, crie uma classe `AlunoMonitor`, que herda da classe `Aluno` e possui um atributo privado `bonus`, representando um acréscimo na média final do aluno. Essa classe deve possuir construtor padrão, construtor com parâmetros, métodos `setBonus`, `getBonus`, `calcularMediaFinal`, `getSituacaoFinal` e `imprimirMonitor`. O método `setBonus` deve lançar uma exceção caso o bônus seja menor que 0 ou maior que 1. No programa principal, crie um vetor com **2 alunos comuns** e também um objeto do tipo `AlunoMonitor`; preencha suas provas, mostre os dados de todos, calcule suas médias e informe qual deles teve o melhor desempenho.

Questão 2 — Controle de estoque com herança e matriz de produtos

Crie um programa em C++ para controlar o estoque de uma loja que possui **2 filiais** e trabalha com **3 produtos perecíveis**. O sistema deverá possuir uma classe `Produto`, contendo os atributos privados `codigo`, `nome` e `preco`. Essa classe deve possuir construtor padrão, construtor com parâmetros, métodos `setCodigo`, `setNome`, `setPreco`, `getCodigo`, `getNome` e `getPreco`. O método `setCodigo` deve lançar uma exceção do tipo `invalid_argument` caso o código seja menor ou igual a zero, o método `setNome` deve lançar `invalid_argument` caso o nome esteja vazio e o método `setPreco` deve lançar `domain_error` caso o preço seja negativo. Em seguida, crie uma classe `ProdutoPerecivel`, que herda de `Produto` e adiciona o atributo privado `diasValidade`.

A classe `ProdutoPerecivel` deve possuir construtor padrão, construtor com parâmetros, métodos `setDiasValidade`, `getDiasValidade`, `estaVencido` e `imprimir`. O método `setDiasValidade` deve lançar `domain_error` caso a quantidade de dias seja negativa, e o método `estaVencido` deve retornar verdadeiro caso o produto tenha 0 dias de validade. Crie também uma classe `Loja`, contendo um vetor com exatamente **3 produtos perecíveis**, um vetor de controle para saber se cada produto foi cadastrado e uma matriz de inteiros **2x3**, em que cada linha representa uma filial e cada coluna representa a quantidade em estoque de um produto. A classe `Loja` deve possuir métodos para cadastrar produtos, adicionar estoque, vender produtos e imprimir um relatório. O programa deve lançar `out_of_range` caso a filial ou o produto informado estejam fora do intervalo permitido, `invalid_argument` caso a quantidade seja menor ou igual a zero, `logic_error` caso tente usar um produto ainda não cadastrado, `domain_error` caso tente vender um produto vencido e `runtime_error` caso tente vender uma quantidade maior que o estoque disponível. No programa principal, cadastre produtos, realize operações válidas e inválidas e trate as exceções usando vários blocos `catch`, respeitando a hierarquia das exceções.

Questão 3 — Cadastro de livros com vetor de objetos dinâmico

Crie um programa em C++ para controlar o cadastro de livros de uma biblioteca. O sistema deverá possuir uma classe `Data`, contendo os atributos privados `dia`, `mes` e `ano`, e uma classe `Livro`, contendo os atributos privados `codigo`, `titulo`, `autor` e um objeto da classe `Data`, representando a data de publicação do livro. A classe `Data` deve possuir construtor padrão, construtor com parâmetros, métodos `setData`, `getDia`, `getMes`, `getAno`, `ehAnteriorA` e `imprimir`. O método `setData` deve lançar uma exceção caso a data seja inválida, como dia menor que 1, mês fora do intervalo de 1 a 12 ou dia incompatível com o mês informado.

A classe `Livro` deve possuir construtor padrão, construtor com parâmetros, métodos `setCodigo`, `setTitulo`, `setAutor`, `setDataPublicacao`, `getCodigo`, `getTitulo`, `getAutor`, `getDataPublicacao` e `imprimir`. O método `setCodigo` deve lançar exceção caso o código seja menor ou igual a zero, e os métodos `setTitulo` e `setAutor` devem lançar exceção caso recebam uma string vazia. No programa principal, pergunte ao usuário quantos livros deseja cadastrar, aloque dinamicamente um vetor de objetos da classe `Livro` com esse tamanho, cadastre todos os livros, imprima os dados cadastrados e informe qual é o livro mais antigo. Ao final do programa, libere a memória alocada.